An ORM is an object-oriented way of representing different data elements and the relationships between them. Each row in a database table gets expressed as an object. Instead of thinking about the Book table, you think about the `Book` class. An ORM lets you query a specific book as well as link to the book's associated author and publisher, which themselves are objects. Django automatically creates reverse relationships, so if you have an `Author` object, you can query all the author's books without having to write any additional code. The relationships between the `Book`, `Author`, and `Publisher` objects are shown in figure 1.7, along with their corresponding database table representations.

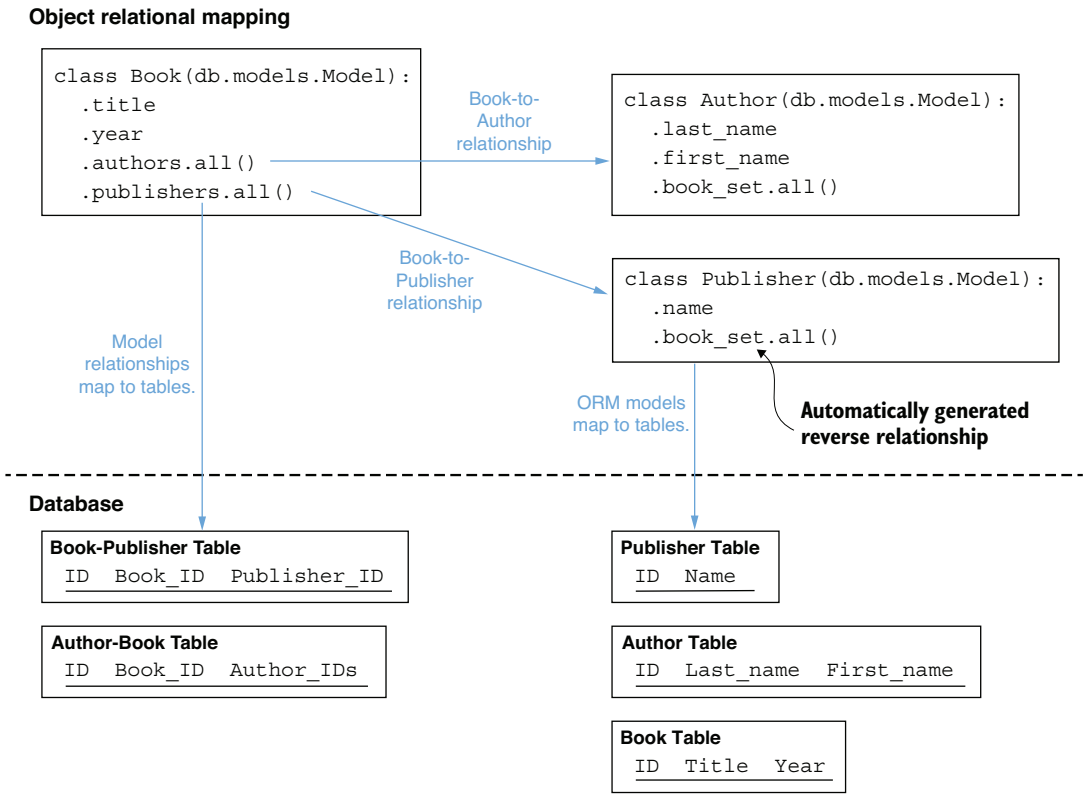


Figure 1.7 ORM Models are data objects that map to underlying item and relationship tables

Django's ORM allows you to define data models that map to tables in a database. You create rows in those tables by creating and saving objects. You can create, read, update, and delete data using the ORM, never having to touch the database itself.

Django's ORM comes with support for multiple databases, including PostgreSQL, MariaDB, MySQL, Oracle, and SQLite. There are also many third-party plugins for other databases. There are even plugins for non-relational databases, like the